

Full Text Search on SQL Server Part 4

By Don Schlichting

Introduction

This article concentrates on using Full Text Search to query text located inside Microsoft Office documents. In previous articles, Microsoft Search was introduced as an add-on service to enable advanced text queries. Catalogs, the physical storage units for search, were created and indexed. The TSQL keywords CONTAINS, FORMSOF, and INFLECTIONAL were used to query the newly created Catalogs. Population Schedules along with Change Tracking options were discussed as methods to keep the Catalogs up to date with the underlying database. This is the forth and final article in the Full Text Search series.

Binary Documents

Full Text Search has the ability to index several types of binary data. Included are Microsoft Word, Excel and PowerPoint documents, along with plain text, XML, and HTML pages. Several steps must take place in order for documents to be searchable. First, a database to store the documents must be created. Then, we will create a Catalog to index the new database, followed by uploading documents into SQL Server. At that point, SQL will be ready to receive queries against the documents.

Database Creation

Step 1 is to create a table for document storage. From Query Analyzer, execute the following TSQL. The table created will become the target for uploaded documents. Remember, the SQL machine where this table is created must be running Windows Server, not workstation. In addition, the Microsoft Search Engine must be installed (see Part 1 for install instructions), because Search is managed by its own service rather than the SQL Server Engine.

```
CREATE TABLE [dbo].[Docs] (  
    [DocID] [int] IDENTITY (1, 1) NOT NULL ,  
    [Type] [varchar] (50) ,  
    [Title] [varchar] (200),  
    [Document] [image] NULL  
) ON [PRIMARY] TEXTIMAGE_ON [PRIMARY]  
GO  
  
ALTER TABLE [dbo].[Docs] WITH NOCHECK ADD  
    CONSTRAINT [PK_Docs] PRIMARY KEY CLUSTERED  
    (  
        [DocID]  
    ) ON [PRIMARY]  
GO
```

Several items are worth noting. First, Full Text Search requires a Primary Key. The search engine will find the Key of the records matching your query, and then pass these Keys to the SQL engine completion of the query. The image field will store the actual document. A "Type" field is required for document search to function. The name and data type of this

field is not important. What matters are that a field exists that will tell the Search Engine what type of document parser to use (Word, Excel, Text, etc) on the row. The document extension "xls", "doc", or "txt" will be stored here.

Catalog Creation

With the table created, the next step is to create a new Catalog. From Enterprise Manager, right click the Docs table and select "Full Text Index Table," then "Define Full Text Indexing on a Table." You must be on the server to perform this operation. A SQL Server registration from your workstation will not work. Click Next on the Welcome page. PK_Docs should now appear as the only index option, click Next. On the Columns page, select Title and Document to search on. Because Document is an image field, Search wants to know what field describes the document parser type to use. To the right of Document, select Type as the "Document type column." Click Next.

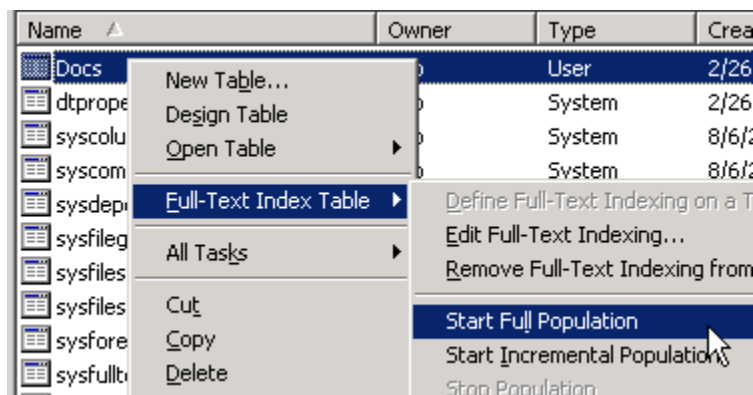
Full-Text Indexing Wizard

Select Table Columns
Select the character-based or image-based columns you want to be eligible for full-text queries.

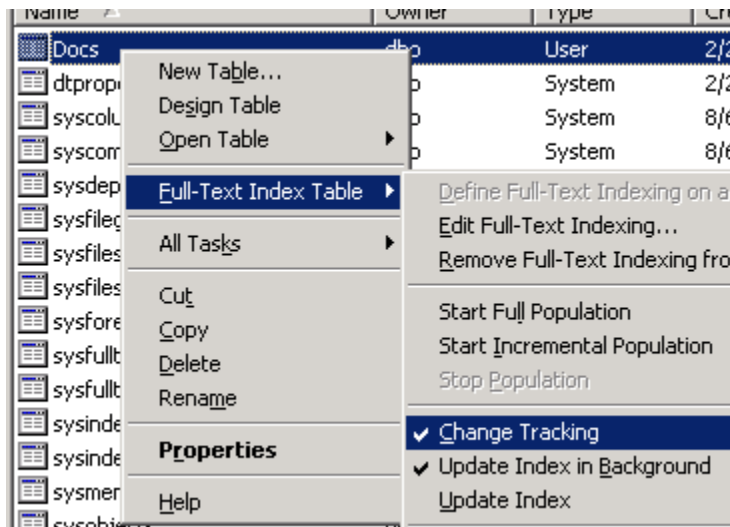
Available columns	Language for Word Breaker	Document type column
☐ Type		
☒ Title		
☒ Document		Type

Create a new Catalog called DocTest. Click Next. On the schedule page, leave all of the options blank and click next. We will use change tracking to keep the Catalog up to date. Click Next, then Finish. The wizard should confirm successful Catalog creation.

Let's do an initial population from Enterprise Manager. Right click the Docs table and select "Full Text Index Table," then "Start Full Population."



To keep the Index up to date in the future, enable Change Tracking. Change Tracking will update the index each time a new document is added or changed in the Docs table. Right click Docs, select "Full Text Index Table," then "Change Tracking" and "Update Index in Background." Make sure to select both.



The table and catalog are now complete. The next step is to move documents into the database.

File Upload

This example uses a Visual Studio Dot Net application written in C#.

With the database complete, we will start inserting documents into the table. This will be accomplished by creating an Upload Web Page that will pass the document to a stored procedure for insert into the cataloged table.

The Upload web page has two distinct features. One is that the form tag includes the encoding command "encType="multipart/form-data"". This is required. The other is that we are using an HTML control, FileField, but with the runat set to server.

The Code Behind page converts the stream of the uploaded document into a stored procedure parameter. It also gets the end of the file name and passes it to the database as the file extension or document type.

The only unique job for the stored procedure is to remove the period from the file name. This step could just as easily be handled in the Code Behind page.

```
--rmv the period
SET @Type = RIGHT(@Type, 3)
SET @Type = LOWER(@Type)
```

I have used the upload page to insert three purchase orders that were originally done in Microsoft Word. A select * verifies the documents where inserted into the database.

```
SELECT * FROM docs
```

	DocID	Type	Title	Document
1	4	doc	Workstation Purchases	OxD0CF11E0A1B11AE
2	5	doc	Barcode Purchases	OxD0CF11E0A1B11AE
3	6	doc	Server Purchases	OxD0CF11E0A1B11AE

The Workstation purchase order included a USB hub. To test our Catalog, I queried for the key word "hub."

```
SELECT *  
FROM docs  
WHERE CONTAINS(Document, ' "hub" ')
```

	DocID	Type	Title	Document
1	4	doc	Workstation Purchases	OxD0CF11E0A1B1

The document containing the key word "hub" is successfully returned.

FREE TEXT TABLE

To receive a numeric rating indicating how closely the returned string is to the search phase, the TSQL key word FREETEXTTABLE is used. FREETEXTTABLE will return a value called RANK, indicating how closely the return is. 1000 is the highest rank. FREETEXTTABLE also returns a column called Key. Key is the primary key of our table. To receive back both RANK and row fields, the FREETEXTTABLE is used like a normal table and joined.

```
SELECT f.*, d.*  
FROM Docs d INNER JOIN FREETEXTTABLE(Docs, *, ' "hub" ') f  
ON d.DocID = f.[Key]
```

Conclusion

Full Text Searching can add advanced string searching capabilities to your application. The ability to schedule Catalog updates, or use live real time background updates, offers several maintenance options depending on the workload of the server. Catalogs require setup and configuration, but are worth the effort.